```python
# Importamos las bibliotecas necesarias
import cv2  # OpenCV: procesamiento de imágenes y captura de video
import mediapipe as mp  # MediaPipe: detección y seguimiento de manos
import numpy as np  # NumPy: operaciones matemáticas con matrices y
vectores
import pyautogui  # pyautogui: controla el mouse y el teclado del sistema

# Inicializamos herramientas de MediaPipe
mp_drawing = mp.solutions.drawing_utils  # Utilidades para dibujar los
landmarks
mp_hands = mp.solutions.hands  # Solución de MediaPipe para detección de
manos

# Capturamos la cámara
cap = cv2.VideoCapture(0)  # Abre la cámara con índice 0 (por defecto)

# Obtenemos la resolución de la pantalla
SCREEN_W, SCREEN_H = pyautogui.size()  # Asigna ancho y alto de la pantalla
del sistema operativo

# Definimos colores RGB para elementos visuales
color_mouse_pointer = (255, 0, 255)  # Color del puntero virtual del mouse
color_base = (255, 0, 112)  # Color del punto base de la palma
color_index = (255, 198, 82)  # Color del dedo índice

# Función para calcular la distancia Euclidiana entre dos puntos (x1, y1) y
(x2, y2)
def calculate_distance(x1, y1, x2, y2):
    return np.linalg.norm(np.array([x1, y1]) - np.array([x2, y2]))

# Función que detecta si el dedo índice está "bajado" (gesto de pinza para
hacer clic)
def detect_finger_down(hand_landmarks, img):
    """
    Esta función determina si el dedo índice está lo suficientemente cerca
del centro de la palma
    como para considerarse bajado (pinch) y dibuja los puntos en la imagen.
    """
    h, w, _ = img.shape  # Obtiene alto, ancho y canales de la imagen

    # Coordenadas del centro de la palma (landmark 0) y base del dedo medio
(landmark 9)
    x_base1 = int(hand_landmarks.landmark[0].x * w)
    y_base1 = int(hand_landmarks.landmark[0].y * h)
    x_base2 = int(hand_landmarks.landmark[9].x * w)
    y_base2 = int(hand_landmarks.landmark[9].y * h)

    # Coordenadas del dedo índice (landmark 8)
    x_index = int(hand_landmarks.landmark[8].x * w)
    y_index = int(hand_landmarks.landmark[8].y * h)

    # Calculamos distancias entre los puntos
```

```
    d_base = calculate_distance(x_base1, y_base1, x_base2, y_base2)  # Palma ↔ base del dedo medio
    d_base_index = calculate_distance(x_base1, y_base1, x_index, y_index)  # Palma ↔ dedo índice

    # Verificamos si el índice está "bajado" (distancia menor)
    down = d_base_index < d_base

    # Elegimos colores dependiendo del estado del dedo índice
    c_base = (255, 0, 255) if down else color_base
    c_index = (255, 0, 255) if down else color_index

    # Dibujamos los puntos y líneas en la imagen para visualización
    cv2.circle(img, (x_base1, y_base1), 5, c_base, 2)  # Punto en el centro de la palma
    cv2.circle(img, (x_index, y_index), 5, c_index, 2)  # Punto en el dedo índice
    cv2.line(img, (x_base1, y_base1), (x_base2, y_base2), c_base, 3)  # Línea base-palma
    cv2.line(img, (x_base1, y_base1), (x_index, y_index), c_index, 3)  # Línea palma-índice

    return down  # Devuelve True si el índice está bajado (pinch)

# Iniciamos el modelo de detección de manos de MediaPipe
def main():
    with mp_hands.Hands(
        static_image_mode=False,  # Se usa para video continuo (no imágenes individuales)
        max_num_hands=1,  # Máximo una mano detectada
        min_detection_confidence=0.5  # Nivel mínimo de confianza requerido
    ) as hands:

        while True:  # Bucle principal
            ret, frame = cap.read()  # Captura un frame de la cámara
            if ret:
                frame = cv2.flip(frame, 1)  # Invierte el frame horizontalmente (efecto espejo)
                height, width, _ = frame.shape  # Obtiene las dimensiones del frame

                # Convierte BGR a RGB y procesa el frame con MediaPipe
                results = hands.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

                if results.multi_hand_landmarks:  # Si se detectan manos
                    for hand_landmarks in results.multi_hand_landmarks:

                        # Coordenadas normalizadas de la base del dedo medio (landmark 9)
                        x_norm = hand_landmarks.landmark[9].x
                        y_norm = hand_landmarks.landmark[9].y

                        # Mapeamos coordenadas normalizadas a coordenadas
```

```
de pantalla
                        x_screen = np.interp(x_norm, [0, 1], [0, SCREEN_W])
                        y_screen = np.interp(y_norm, [0, 1], [0, SCREEN_H])

                        # Movemos el cursor del mouse a la posición de la
mano
                        pyautogui.moveTo(int(x_screen), int(y_screen))

                        # Detectamos gesto de clic (pinch) y ejecutamos
clic si aplica
                        if detect_finger_down(hand_landmarks, frame):
                            pyautogui.click()

                        # Dibujamos el puntero del mouse sobre el frame
                        cv2.circle(frame, (int(x_norm * width), int(y_norm
* height)), 10, color_mouse_pointer, 3)
                        cv2.circle(frame, (int(x_norm * width), int(y_norm
* height)), 5, color_mouse_pointer, -1)

                # Mostramos el frame procesado en una ventana
                cv2.imshow("mouse", frame)
                key = cv2.waitKey(1)  # Espera por 1 ms para capturar una
tecla
                if key == ord("q"):  # Si la tecla presionada es "q", se
sale del bucle
                    break
            else:
                print("No se pudo leer el frame de la cámara.")  # Error al
capturar el frame
                break

    cap.release()  # Libera la cámara
    cv2.destroyAllWindows()  # Cierra todas las ventanas de OpenCV

# Ejecutamos la función principal
main()
```